Unit 06: Type Conversion

CONTENTS

Objectives

After studying this unit, you will be able to:

- Recognize the type conversions
- Describe the basic type to class type
- Explain the class type to basic type
- Discuss the class type to another type

Introduction

It is the transformation of one type into another. Type casting, in other terms, is the process of transforming an expression of one type into another.

Conversion of variables from one type to another are known as type conversion. Type conversions ultimate aim is to make variables of one data type work with variables of another data type.

Type conversions can be used to force the correct type of mathematical computation needed to be performed.

Depending on whether the type conversion is ordered by the programmer or the compiler, it can be explicit or implicit. When a programmer wants to go around the compiler's typing system, he or she uses explicit type conversions (casts); however, the programmer must use them appropriately to succeed. Problems that the compiler avoids may develop, such as when the processor requires data of a certain type to be stored at specific addresses or when data is truncated because a data type on a given platform does not have the same size as the original type. Explicit type conversions between objects of different kinds result in difficult-to-read code at best.

6.1 Type Conversion

Constants and variables in a mixed expression are of distinct data kinds. According to certain rules, assignment operations cause automatic type conversion between the operands.

The data type to the right of an assignment operator is transformed to the data type of the variable on the left automatically.

Example

int a = 45;

float b= 1253.25;

a=b;

This converts float variable b to an integer before its value assigned to a. The type conversion is automatic as far as data types involved are built in types. We can also use the assignment operator in case of objects to copy values of all data members of right hand object to the object on left hand. The objects in this case are of same data type.

Different types of Type Conversion

1. Implicit type conversion
2. Explicit type conversion

Implicit type conversion

Done by the compiler on its own, without any external trigger from the user. Generally takes place when in an expression more than one data type is present. In such condition type conversion takes place to avoid loss of data.

Lab Excersise

```
//Program
#include <iostream>
using namespace std;
int main()
{
        int x = 100;
        char y = 'a';
        x = x + y;
float z = x + 1.0;
        cout << "x = " << x << endl
                << "y = " << y << endl
                << "z = " << z << endl;
        return 0;
}
```

Output

```
x = 197
y = a
z = 198

Process returned 0 (0x0)   execution time : 0.030 s
Press any key to continue.
```

Explicit type conversion

This process is user defined, also called type casting.

Lab Exercise

```
//Program
#include <iostream>
using namespace std;
int main()
{
   double x = 25.5;
   int sum = (int)x + 1;
  cout << "Sum = " << sum;
   return 0;
}
```

Output

```
Sum = 26
Process returned 0 (0x0)   execution time : 0.099 s
Press any key to continue.
```

There are three types of situations that arise where data conversion are between incompatible types. Possible Type Conversions in C++

1. Conversion from basic type to class type
2. Conversion from class type to basic type
3. Conversion from one class type to another class type

6.2 Basic Type to Class Type

A constructor was used to build a matrix object from an int type array. Similarly, we used another constructor to build a string type object from a char* type variable. In these examples constructors performed a defect type conversion from the argument's type to the constructor's class type

Consider the following constructor:

```
string::string(char*a)
{
length = strlen(a);
name=new char[len+1];
strcpy(name,a);
}
```

This constructor builds a string type object from a char* type variable a. The variables length and name are data members of the class string. Once you define the constructor in the class string, it can be used for conversion from char* type to string type.

Example

```
string s1, s2;
```

char* name1 = "Good Morning";

char* name2 = "STUDENTS" ;

s1 = string(name1);

s2 = name2;

The program statement

s1 = string (name1);

first converts name1 from char* type to string type and then assigns the string type values to the object s1. The statement

s2 = name2;

performs the same job by invoking the constructor implicitly.

Consider the following example

class time

{

int hours;

int minutes;

public:

time (int t) II constructor

{

hours = t / 60; //t is inputted in minutes

minutes = t % 60; .

}

};

In the following conversion statements:

time T1; //object T1 created

int period = 160;

T1 = period; //int to class type

The object T1 is created. The variable period of data type integer is converted into class type time by invoking the constructor. After this conversion, the data member hours of T1 will have value 2 arid minutes will have a value of 40 denoting 2 hours and 40 minutes.

In both the examples, the left-hand operand of = operator is always a class object. Hence, we can also accomplish this conversion using an overloaded = operator.

 Lab exercise

// Program - Using Constructor

#include<iostream>

using namespace std;

class Time

{

int hrs,min;

```
public:
Time(int t)
{
cout<<"Basic Type to Class Type Conversion...\n";
hrs=t/60;
min=t%60;
}
void show();
};
void Time::show()
{
cout<<hrs<< ": Hours(s)" <<endl;
cout<<min<< " Minutes" <<endl;
}
int main()
{
int duration;
cout<<"\nEnter time duration in minutes";
cin>>duration;
Time t1(duration);
t1.show();
return 0;
}
```

Output

```
Enter time duration in minutes850
Basic Type to Class Type Conversion...
14: Hours(s)
10 Minutes

Process returned 0 (0x0)   execution time : 1.778 s
Press any key to continue.
```

Lab exercise

```
// Program - Using Operator
#include<iostream>
using namespace std;
class Time
{
```

```
int hrs,min;
public:
void display()
{
cout<<hrs<< ": Hour(s)\n";
cout<<min<<": Minutes\n";
}
void operator =(int t)
{
cout<<"\nBasic Type to Class Type Conversion...\n";
hrs=t/60;
min=t%60;
}
};
int main()
{
Time t1;
int duration;
cout<<"Enter time duration in minutes";
cin>>duration;
cout<<"object t1 overloaded assignment..."<<endl;
t1=duration;
t1.display();
cout<<"object t1 assignment operator 2nd method..."<<endl;
t1.operator=(duration);
t1.display();
return 0;
}
```

Output

```
Enter time duration in minutes521
object t1 overloaded assignment...

Basic Type to Class Type Conversion...
8: Hour(s)
41: Minutes
object t1 assignment operator 2nd method...

Basic Type to Class Type Conversion...
8: Hour(s)
41: Minutes

Process returned 0 (0x0)   execution time : 4.971 s
Press any key to continue.
```

Notes

- In this conversion source type is basic type and the destination type is class type.
- Basic data type is converted into class type.

6.3 Class Type to Basic Type

The constructor functions do not support conversion from a class to basic type. C++ allows us to define a overloaded casting operator that convert a class type data to basic type. The general form of an overloaded casting operator function, also referred to as a conversion function, is:

```
operator typename()
{
//Program statement
}
```

This function converts a class type data to type name. For example, the operator double() converts a class object to type double, in the following conversion function:

```
vector:: operator double()
{
double sum = 0;
for(int I = 0; i<size; i++)
sum = sum + v[i] * v[i ]; //scalar magnitude
return sqrt(sum);
}
```

Did you know?

Conversion function must be a class member.

Conversion function must not specify the return value even though it returns the value.

Conversion function must not have any argument.

In the string example discussed earlier, we can convert the object string to char* as follows:

```
string::operator char*()
{
retum(str);
}
```

Did you know?

Dynamic_cast can be used only with pointers and references to objects. Its purpose is to ensure that the result of the type conversion is a valid complete object of the requested class.

Lab exercise

```
// Program

#include<iostream>
using namespace std;
class Time
{
int h,m;
public:
Time(int a,int b)
{
h=a;
m=b;
}
operator int()
{
cout<<"\nClass Type to Basic Type Conversion...";
return(h*60+m);
}
~Time()
{
cout<<"\nDestructor called..."<<endl;
}
};
int main()
{
int h,m,duration;
cout<<"\nEnter Hours ";
cin>>h;
cout<<"\nEnter Minutes ";
cin>>m;
Time t(h,m);
duration = t;
cout<<"\nTotal Minutes are "<<duration;
cout<<"\n2nd method operator overloading ";
duration = t.operator int();
cout<<"\nTotal Minutes are "<<duration;
return 0;
}
```

Output

```
Enter Hours 850

Enter Minutes 1200

Class Type to Basic Type Conversion...
Total Minutes are 52200
2nd method operator overloading
Class Type to Basic Type Conversion...
Total Minutes are 52200
Destructor called...

Process returned 0 (0x0)   execution time : 4.923 s
Press any key to continue.
```

Notes

Class type to basic type conversion requires special casting operator function for class type to basic type conversion. This is known as the conversion function.

6.4 Class Type to another Class Type

We have just seen data conversion techniques from a basic to class type and a class to basic type. But sometimes we would like to convert one class data type to another class type.

Example

Obj1 = Obj2 ; //Obj1 and Obj2 are objects of different classes.

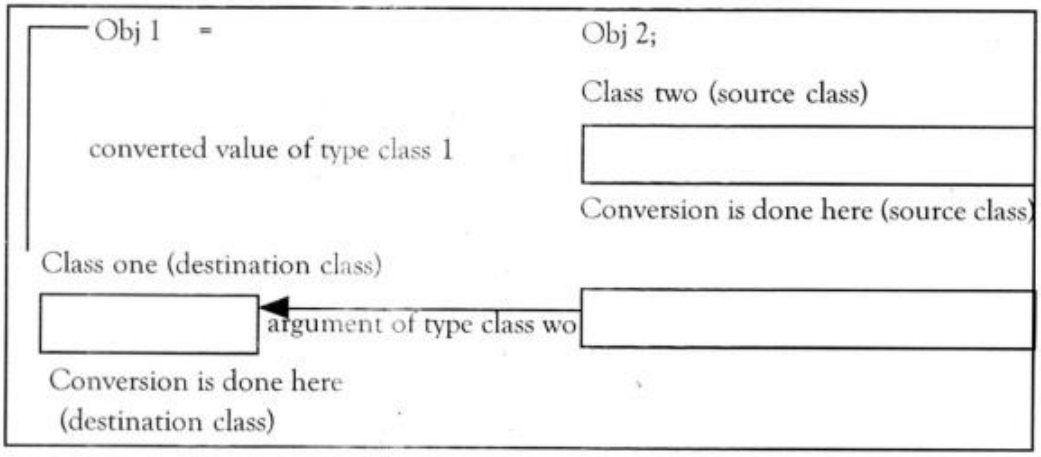
Obj1 is an object of class one and Obj2 is an object of class two. The class two type data is converted to class one type data and the converted value is assigned to the Obj1. Since the conversion takes place from class two to class one, two is known as the source and one is known as the destination class.

Such conversion between objects of different classes can be carried out by either a constructor or a conversion function. Which form to use, depends upon where we want the type-conversion function to be located, whether in the source class or in the destination class.

We studied that the casting operator function

Operator typename()

The following Figure illustrates the above two approaches.



The following Table summarizes all the three conversions. It shows that the conversion from a class to any other type (or any other class) makes use of a casting operator in the source class. To perform the conversion from any other type or class to a class type, a constructor is used in the destination class.

Conversion	Conversion takes place in	
	Source class	Destination class
Basic → class	Not applicable	Constructor
Class → Basic	Casting operator	Not applicable
Class → class	Casting operator	Constructor

When a conversion using a constructor is performed in the destination class, we must be able to access the data members of the object sent (by the source class) as an argument.

Lab Exercise

```
// Program – Using Constructor
#include<iostream>
using namespace std;
class Time
{
int hrs,min;
public:
Time(int h,int m)
{
hrs=h;
min=m;
```

```
}
Time()
{
cout<<"\n Time's Object Created";
}

int getMinutes()
{
int tot_min = ( hrs * 60 ) + min ;
return tot_min;
}

void display()
{
cout<<"Hours: "<<hrs<<"\n";
cout<<" Minutes : "<<min <<"\n";
}
};
class Minute
{
int min;
public:
Minute()
{
min = 0;
}
void operator=(Time T)
{
min=T.getMinutes();
}
void display()
{
cout<<"\n Total Minutes : " <<min<<"\n";
}
};
int  main()
{
Time t1(1,20);
t1.display();
Minute m1;
m1.display();
```

```
m1 = t1;
t1.display();
m1.display();
return 0;
}
```

Output

```
Hours: 1
 Minutes : 20

 Total Minutes : 0
Hours: 1
 Minutes : 20

 Total Minutes : 80

Process returned 0 (0x0)   execution time : 0.015 s
Press any key to continue.
```

 Lab Exercise

```
//Program – Using conversion function
#include<iostream>
using namespace std;
class inventory1
{
int ino,qty;
float rate;
public:
inventory1(int n,int q,float r)
{
ino=n;
qty=q;
rate=r;
}
int getino()
{
return(ino);
}
float getamt()
{
return(qty*rate);
}
```

```
void display()
{
cout<<"\nino = "<<ino<<" qty = "<<qty<<" rate = "<<rate;
}
};
class inventory2
{
int ino;
float amount;
public:
void operator=(inventory1 I)
{
ino=I.getino();
amount=I.getamt();
}
void display()
{
cout<<"\nino = "<<ino<<" amount = "<<amount;
}
};
int main()
{
inventory1 I1(101,50,45);
inventory2 I2;
I2=I1;
I1.display();
I2.display();
}
```

```
ino = 101 qty = 50 rate = 45
ino = 101 amount = 2250
Process returned 0 (0x0)   execution time : 0.224 s
Press any key to continue.
```

Summary

- A type conversion may either be explicit or implicit, depending on whether it is ordered by the programmer or by the compiler. Explicit type conversions (casts) are used when a programmer want to get around the compiler's typing system; for success in this endeavour, the programmer must use them correctly.
- Used another constructor to build a string type object from a char* type variable.

- The general form of an overloaded casting operator function, also referred to as a conversion function, is:

```
operator typename()
{
//Program statement
}
```

- Which form to use, depends upon where we want the type-conversion function to be located, whether in the source class or in the destination class.

Keywords

Implicit Conversion: An implicit conversion sequence is the sequence of conversions required to convert an argument in a function call to the type of the corresponding parameter in a function declaration.

Explicit Conversion:

Operator Typename(): Converts the class object of which it is a member to typename.

Self Assessment

1. Following program is an example of _____ conversion.

```
#include <iostream>
using namespace std;
int main()
{
    int x = 100;
    char y = 'a';
    x = x + y;
float z = x + 1.0;
    cout << "x = " << x << endl
      << "y = " << y << endl
      << "z = " << z << endl;
    return 0;
}
```

A. Implicit
B. Explicit
C. Both
D. None of Above

2. What is type casting?

A. Converting one function into another
B. Converting one data type into another
C. Converting operator type to another type

D. None of them

3. Choose the correct syntax for explicit conversion.

A. Explicit (type)

B. (type) expression;

C. Expression (explicit)

D. None of Above

4. Who carries out implicit type casting?

A. The Micro Controller

B. The Compiler

C. The Programmer

D. The User

5. Who initiates explicit type casting?

A. The Micro Controller

B. The Compiler

C. The Programmer

D. The User

6. What will be the data type of the result of the following operation?

(float)a * (int)b / (long)c * (double)d

A. int

B. long

C. float

D. double

7. When double is converted to float, the value is?

A. Truncated

B. Rounded

C. Depends on the compiler

D. Depends on the standard

8. Which of the following type conversion is not possible in C++?

A. Basic to Class type

B. Class to Basic type

C. One Class to another class type

D. Inheritance to inheritance

9. Which of the following is correct statement for class to basic type conversion?

A. Class type to basic type conversion never performed

B. In this conversion source type is class type and the destination type is basic type.

C. Class type to basic type conversion acts like data type

D. None of above

10. Conversion function _____.

A. must be a class member

B. must not have any argument

C. All of above

D. None of above

11. Conversion function must not specify the return value even though it returns the value.

A. True

B. False

12. To convert from a user defined class to a basic type, you would most likely use.

A. Built-in conversion function

B. A one-argument constructor

C. A conversion function that's a member of the class

D. An overloaded '='operator

13. How many ways to perform conversion from one class to another class can perform?

A. 4

B. 2

C. 3

D. 1

14. ____ refers to the process of changing the data type of the value stored in a variable.

A. Type char

B. Type int

C. Type float

D. Type cast

15. Which of the following type-casting have chances for wrap around?

A. From int to float

B. From int to char

C. From char to short

D. From char to int

Answers for Self Assessment

1.	A	2.	B	3.	B	4.	B	5.	C
6.	D	7.	C	8.	D	9.	B	10.	C
11.	A	12.	C	13.	B	14.	D	15.	B

Review Questions

1. What do you mean by type casting? Explain the difference between implicit and explicit type casting in detail.
2. How type conversion occurs in a program. Explain with suitable example.
3. Write a program that demonstrate working of explicit type conversion.

4. The assignment operations cause automatic type conversion between the operand as per certain rules. Describe.
5. Write a program that demonstrate working of implicit type conversion.
6. What is class to another class type conversion?
7. List the situations in which we need class type to basic type conversion.
8. How to convert one data type to another data type in C++. Explain in detail.
9. Write a program which the conversion of class type to basic type conversion.
10. There are three types of situations that arise where data conversion are between incompatible types. What are three situations explain briefly.

Further Readings

E Balagurusamy; Object-oriented Programming with C++; Tata Mc Graw-Hill.

Herbert Schildt; The Complete Reference C++; Tata Mc Graw Hill.

Robert Lafore; Object-oriented Programming in Turbo C++; Galgotia.

Web Links

http://msdn.microsoft.com/en-us/library/wcz57btd(v=vs.80).aspx

http://www.learncpp.com/cpp-tutorial/117-multiple-inheritance/

https://www.codecademy.com/learn/learn-c-plus-plus